SOFTWARE REQUIREMENTS SPECIFICATION

Resume / Job-Match Analyzer

Multi-Agent, Evidence-Grounded Resume Tailoring System

Field	Value
Document Type	SRS and Project Proposal
Project Status	Mentor-approved; revised MVP and future roadmap
Planned Deployment	9-10 August 2026
Final Submission	14 August 2026
Delivery Constraint	No paid models, APIs, hosting plans, or subscriptions
Version	3.0 - Multi-Job Career Intelligence Scope

Prepared for Agentic AI Full-Stack Internship

Executive Summary

The Resume / Job-Match Analyzer is a full-stack multi-agent career intelligence application designed to help job seekers compare one evidence-based candidate profile against multiple job opportunities, rank the opportunities by fit, and tailor the resume for a selected role without fabricating experience. A user uploads or pastes a resume and supplies up to three job descriptions in the MVP. A supervisor agent coordinates job-analysis workers, a cross-job ranking service, an ATS/keyword worker, and a bullet-rewriter worker. Structured outputs are validated by deterministic backend rules and merged into an explainable dashboard showing ranked jobs, matched strengths, wording gaps, evidence gaps, real skill gaps, repeated missing skills, keyword coverage, and safe rewrites. Future releases may enrich the candidate profile from GitHub, user-approved career chats, certificates, and company research. The project demonstrates planning, delegation, parallel execution, tool use, schema-constrained output, validation, synthesis, and human approval while remaining feasible within the August 2026 timeline and a zero-cost technology constraint.

1. Problem Statement

Job applicants frequently submit a generic resume to multiple vacancies even though each job description emphasizes a different combination of skills, tools, responsibilities, seniority, and domain language. Manually tailoring a resume is time-consuming and requires the applicant to interpret requirements, identify genuine evidence in their background, determine missing keywords, and rewrite bullets in a way that is both ATS-friendly and truthful. Existing general-purpose AI tools can produce polished wording, but they may invent achievements, technologies, metrics, or leadership responsibilities that are not supported by the original resume. The proposed system solves this problem by separating the work across specialized AI agents, grounding every recommendation in resume evidence, and applying deterministic validation before presenting the final report.

1.1 Opportunity

- Reduce the time needed to compare a resume with a job description.
- Improve the relevance and clarity of existing resume content.
- Make ATS keyword gaps visible without encouraging false claims.
- Demonstrate a credible, explainable multi-agent workflow rather than a single prompt.
- Provide a practical portfolio project combining frontend, backend, AI orchestration, validation, testing, and deployment.

2. Target Users and Core Use Case

User Type	Need	Expected Value
Primary: Job applicant	Tailor an existing resume for a specific vacancy	Clear match analysis and truthful rewrites
Secondary: Career mentor	Review a candidate's positioning	Structured evidence and prioritized gaps
Secondary: Internship evaluator	Assess technical and agentic-AI capability	Visible planning, agents, validation, and deployed workflow

2.1 Core Use Case

A job applicant provides one resume and up to three target job descriptions. The system extracts and structures the inputs, analyzes each vacancy independently, calculates an explainable match score, and ranks the jobs from strongest to weakest fit. It distinguishes a wording gap, where the candidate has evidence but the resume presents it poorly; an evidence gap, where a claimed skill needs stronger proof; and a real skill gap, where no supporting evidence exists. After the user selects a job, the system proposes

evidence-grounded bullet rewrites and allows the user to accept, reject, or edit each change. The MVP exports approved content in a structured copyable format; generation of a fully reformatted PDF is classified as a future enhancement because exact PDF layout preservation is not reliable within the remaining build window.

3. Product Scope and User Experience

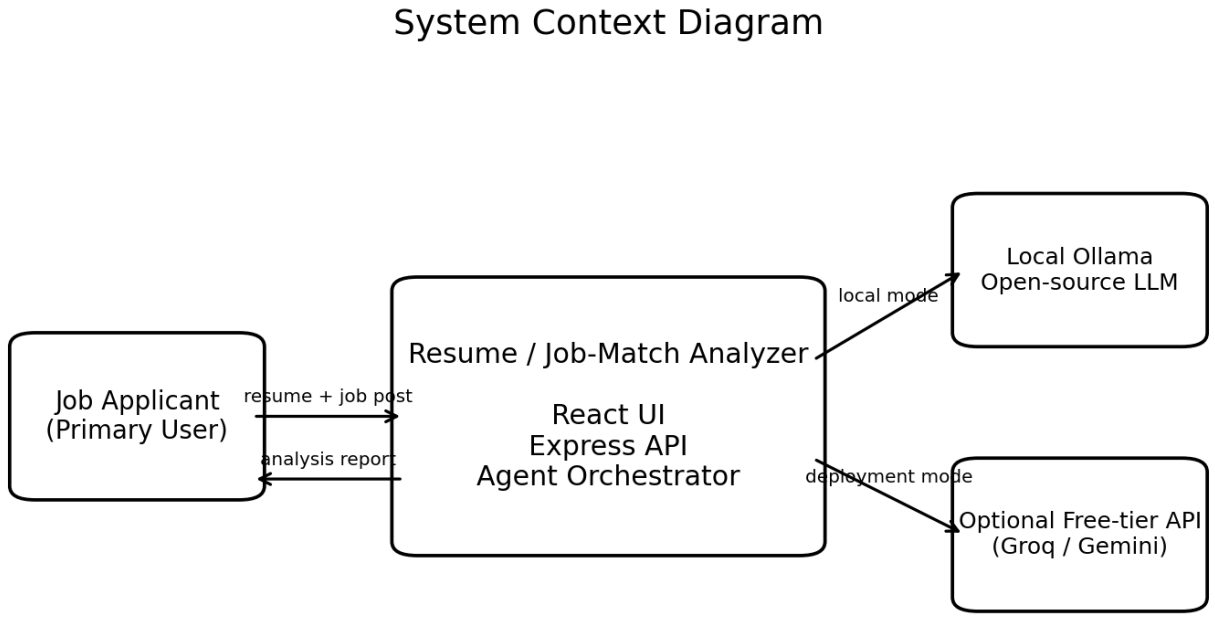

Figure 1. System context: the user interacts with the web application, which selects a zero-cost model path.

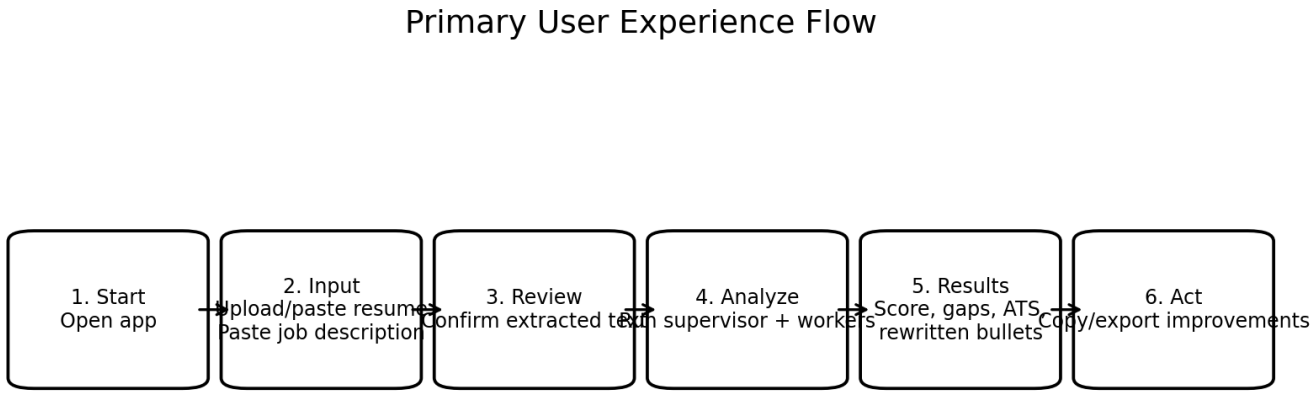

Figure 2. Primary UX flow from input to actionable output.

3.1 Main Screens

Screen	Purpose	Key Elements
Landing / Start	Explain value and begin analysis	Project summary, privacy note, start button
Input	Capture source material	Resume upload/text area, job-description area, validation
Review	Allow correction before AI processing	Extracted resume text, file name, edit controls
Processing	Expose agent activity	Current step, agent status, retry or failure messages
Results Dashboard	Present the complete analysis	Score, strengths, gaps, keywords, rewrites, evidence
History (recommended)	Reopen prior analyses	Job title, date, score, delete/open actions

4. Key AI Features and Agent Primitives

The system will demonstrate agentic AI through explicit primitives rather than merely sending one large prompt to a model.

Agent Primitive	Implementation in This Project	Purpose
Planning	Supervisor produces a structured task plan	Determine required analysis tasks
Delegation	Supervisor assigns tasks to specialized workers	Separate concerns and prompts
Parallel execution	Independent workers run concurrently	Reduce response time
Structured output	Agents return schema-constrained JSON	Enable reliable validation and UI rendering
Tool use	PDF parser, scoring function, validator, persistence layer	Combine LLM reasoning with deterministic software
Grounding	Worker outputs cite resume evidence IDs	Prevent unsupported recommendations
Synthesis	Supervisor merges validated worker outputs	Create one coherent report
Retry / fallback	Failed workers retry once or return partial status	Improve resilience
Guardrails	Post-generation fabrication checks	Protect factual integrity

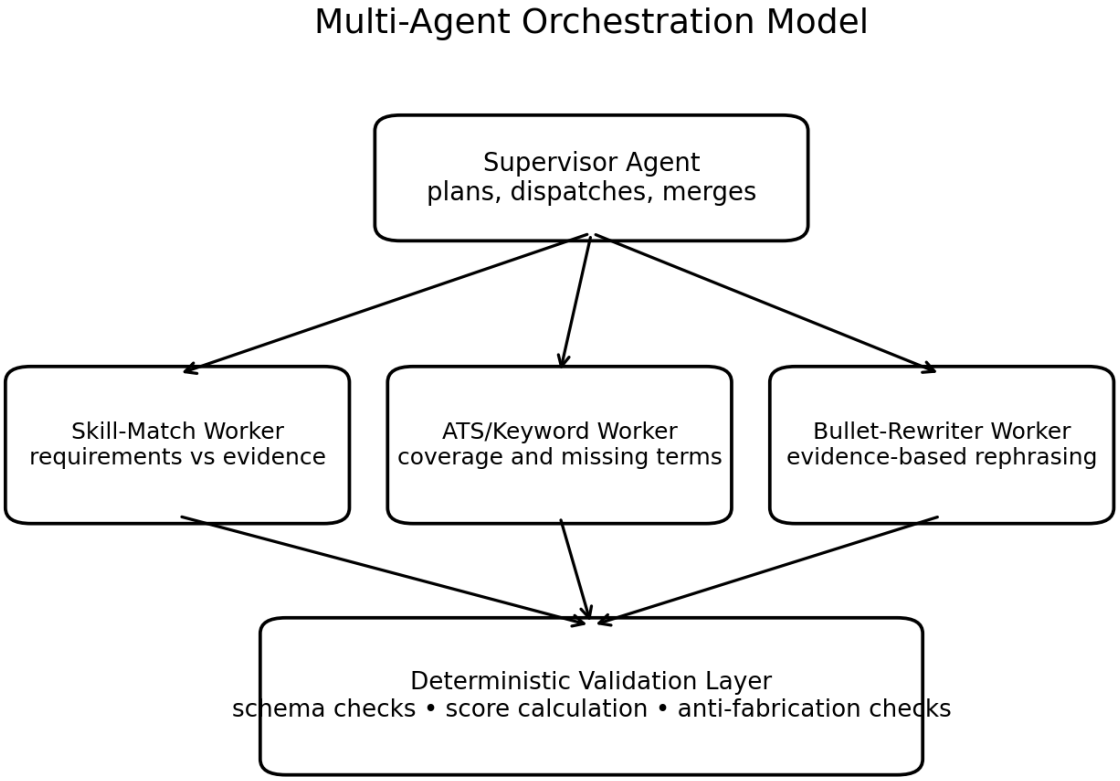

Figure 3. Supervisor-worker architecture with deterministic validation.

4.1 Supervisor Agent

- Reads the normalized resume and job-description objects.
- Creates a plan listing required workers and objectives.
- Dispatches workers, preferably in parallel where dependencies allow.
- Collects successful and failed worker responses.
- Requests correction when structured output is invalid.
- Merges validated findings into the final report.

4.2 Skill-Match Worker

- Classifies job requirements as mandatory, preferred, or contextual.
- Maps each requirement to exact resume evidence where available.
- Labels each requirement matched, partial, missing, or uncertain.
- Produces category-level findings for skills, experience, education, and domain relevance.
- Does not calculate the final score directly; backend logic calculates it from structured findings.

4.3 ATS / Keyword Worker

- Extracts important hard skills, tools, certifications, role language, and repeated terms.
- Measures whether each term appears directly, appears through a synonym, or is absent.
- Ranks keywords by job relevance rather than simple frequency alone.
- Flags terms that should only be added if truthful.

4.4 Bullet-Rewriter Worker

- Selects only resume bullets relevant to the target vacancy.
- Rephrases wording using supported job language and stronger action verbs.

- Preserves factual meaning and does not introduce new technology, metrics, scope, or responsibility.
- Returns original bullet, rewritten bullet, evidence reference, changed keywords, and risk status.

4.5 Anti-Fabrication Validator

- Rejects invented numbers, percentages, monetary values, years, team sizes, and project counts.
- Flags new skills or tools absent from the source resume.
- Flags promotions, leadership, ownership, or certification claims not evidenced.
- Requires every rewrite to reference an original bullet identifier.
- Marks uncertain outputs for user review instead of silently accepting them.

5. System Architecture

The solution follows a layered full-stack architecture. The frontend handles user interaction and visualization. The backend owns file processing, validation, orchestration, scoring, persistence, and AI-provider access. The model is treated as a replaceable dependency behind a provider adapter.

Layer	Responsibilities
Presentation	Forms, validation feedback, progress state, results dashboard, copy/export
API	Request validation, authentication if included, response contracts, error handling
Document Processing	PDF text extraction and normalization
Agent Orchestration	Planning, worker execution, retries, synthesis
Deterministic Services	Scoring, schema validation, evidence checking, risk flags
AI Provider Adapter	Ollama local calls and optional free-tier API calls
Persistence	Analysis history and metadata, if implemented

6. Technology Stack and Zero-Cost Model Strategy

Area	Preferred Technology	Reason
Frontend	React + Vite	Fast development and component-based UI
Styling	Tailwind CSS or standard CSS modules	No paid dependency
Charts	Recharts	Simple score and coverage visuals
Backend	Node.js + Express	Matches internship full-stack skills
Validation	Zod	Schema validation for API and agent outputs
PDF parsing	pdf-parse or pdfjs-dist	Text extraction without paid service
Local LLM	Ollama with Llama 3.1, Qwen2.5, or Gemma	No token cost; works locally
Deployment LLM	Configurable free-tier Groq or Gemini API	Public demo where local Ollama is unreachable
Database	SQLite + Prisma, or MongoDB Atlas free tier	Simple zero-cost persistence
Testing	Vitest/Jest + Supertest	Unit and API testing
Deployment	Free-tier frontend/backend hosting	No paid plan required

7. Data Sources and Integrations

Source / Integration	Required?	Use	Constraint
User resume text	Yes	Primary source of candidate evidence	Must not be fabricated or silently changed
Resume PDF upload	Recommended	Convenient input path	Text-based PDFs only for MVP; no paid OCR
Job-description text	Yes	Target requirements and keywords	User pastes content manually
Ollama local API	Yes for local mode	Open-source model inference	Requires local machine resources
Free-tier AI API	Optional for deployment	Remote inference for public demo	Quota and policy may change
Database	Recommended	Analysis history	Must use free/local option
Job-site scraping	No	Not needed for MVP	Avoid legal, technical, and time risk
Commercial ATS API	No	Not required	Out of scope and potentially paid

8. Interaction and Sequence Design

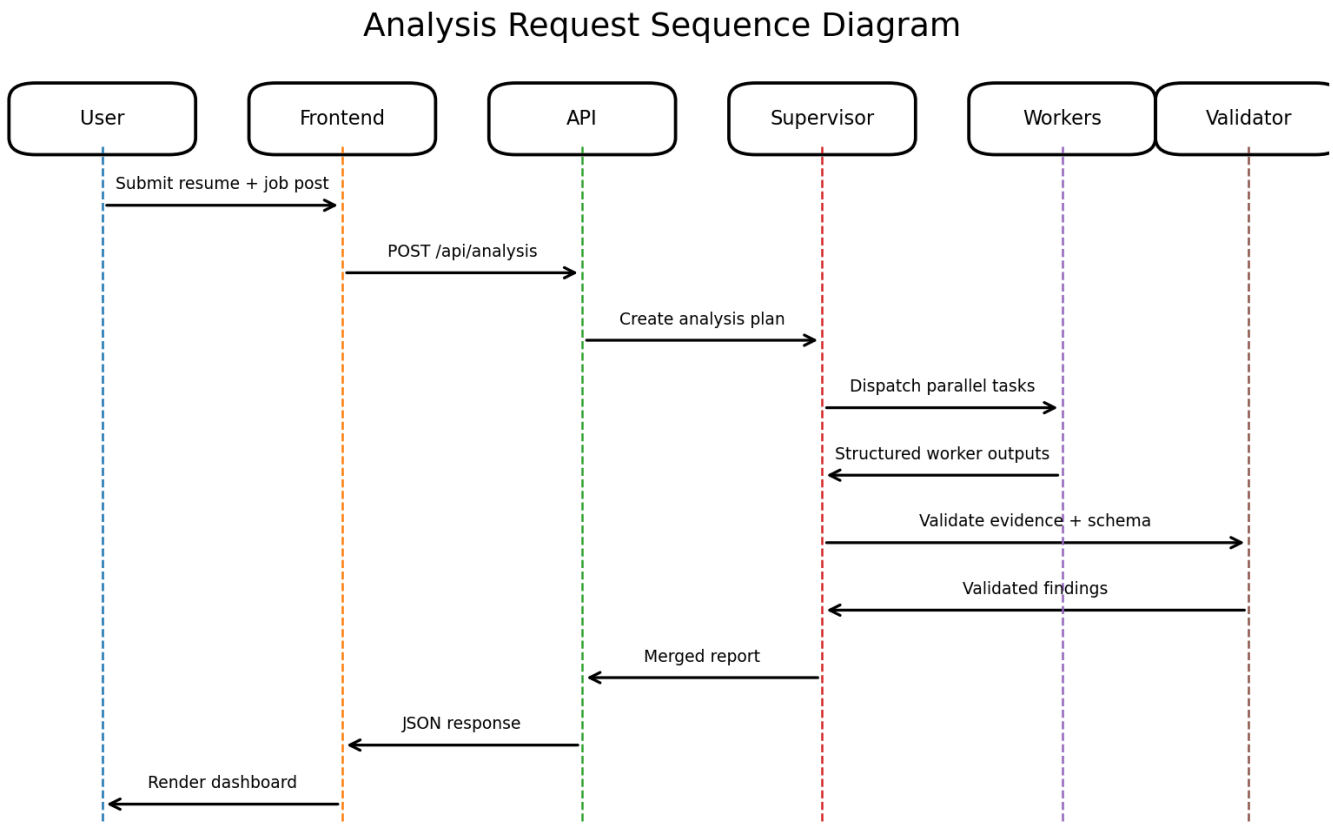

Figure 4. End-to-end analysis sequence from user submission to dashboard rendering.

9. Functional Requirements

ID	Requirement	Specification
FR-01	Resume input	The system shall accept pasted resume text.
FR-02	PDF upload	The system shall accept text-based PDF resumes within a configured size limit.
FR-03	Extraction review	The system shall display extracted text before analysis.
FR-04	Job input	The system shall accept pasted job-description text.
FR-05	Input validation	The system shall reject empty, unsupported, or excessively large input.
FR-06	Planning	The supervisor shall produce a machine-readable analysis plan.
FR-07	Skill analysis	The system shall classify job requirements against resume evidence.
FR-08	Keyword analysis	The system shall identify present, partial, and missing ATS keywords.
FR-09	Bullet rewriting	The system shall rewrite only existing, relevant resume bullets.
FR-10	Evidence mapping	Each match and rewrite shall include a resume evidence reference.
FR-11	Score calculation	Backend code shall calculate an overall score from weighted structured findings.
FR-12	Agent log	The UI shall show each agent's task and status.
FR-13	Partial completion	The system shall return valid partial results when one worker fails.
FR-14	Results dashboard	The UI shall show overview, skills, keywords, rewrites, and recommendations.
FR-15	Copy/export	The user shall be able to copy key results; text/JSON export is recommended.
FR-16	History	The system should store and retrieve prior analyses when persistence is enabled.
FR-17	Delete history	The user should be able to delete a stored analysis.
FR-18	Provider selection	The backend shall select the model provider using environment configuration.
FR-19	Health check	The backend shall provide a health endpoint.
FR-20	Reset	The user shall be able to clear the current analysis and start again.
FR-21	Multiple job inputs	The MVP shall accept between one and three job descriptions for a single resume analysis.
FR-22	Cross-job ranking	The system shall rank submitted jobs using deterministic weighted scores derived from structured evidence.
FR-23	Ranking explanation	Each ranked job shall show score drivers, mandatory gaps, and a recommendation label.
FR-24	Gap classification	The system shall classify each gap as a wording gap, evidence gap, real skill gap, or uncertain.
FR-25	Repeated gap summary	The system shall identify skills repeatedly missing across the submitted jobs.
FR-26	Job selection	The user shall be able to select one ranked job for detailed tailoring.
FR-27	Change approval	The user shall be able to accept, reject, or edit each proposed bullet rewrite.
FR-28	Tailored content output	The MVP shall generate approved, copyable job-specific resume content without overwriting the original file.

13. Non-Functional Requirements

ID	Area	Requirement
NFR-01	Cost	The complete MVP shall use no paid model, API, database, hosting plan, or subscription.
NFR-02	Performance	A normal analysis should complete within a practical live-demo window; target under 60 seconds in API mode and under 120 seconds locally.
NFR-03	Reliability	A single worker failure shall not crash the entire analysis.
NFR-04	Security	Secrets shall be stored in environment variables and excluded from source control.
NFR-05	Privacy	Resume content shall not be written to application logs.
NFR-06	Usability	The core workflow shall be understandable without training.
NFR-07	Responsiveness	The interface shall support common laptop and mobile widths.
NFR-08	Maintainability	Agents, prompts, providers, validators, and routes shall be separated into modules.
NFR-09	Explainability	Scores and suggestions shall be traceable to requirements and resume evidence.
NFR-10	Accessibility	Forms shall have labels, keyboard support, and readable contrast.

14. Testing and Acceptance Criteria

Test Area	Minimum Coverage
Input validation	Empty input, large input, unsupported file, unreadable PDF
Parsing	Standard resume sections, no skills section, unusual headings
Schema validation	Valid JSON, malformed JSON, missing required fields
Scoring	Weight totals, partial matches, score cap, mandatory gap penalty
Anti-fabrication	Invented metrics, new skills, unsupported leadership, changed dates
Agent resilience	One worker timeout, retry success, retry failure, partial report
API	Health, parse, analyze, retrieve, delete
UI	Happy path, loading, error, retry, responsive layout
End-to-end	At least three resume/job combinations

14.1 Revised Product Differentiation

The revised product identity is an evidence-based, multi-job career intelligence system. Its strongest defensible uniqueness is not merely the use of several agents; it is the ability to rank multiple jobs using verified candidate evidence and explain whether a mismatch is caused by weak wording, weak evidence, or a genuine capability gap.

Differentiator	MVP Implementation	User Value
Multi-job opportunity ranking	Compare one resume with up to three job descriptions and rank them by deterministic score.	Helps users decide where to apply first.
Three-way gap intelligence	Classify gaps as wording, evidence, or real skill gaps.	Prevents every absence from being treated as the same problem.

Evidence-grounded tailoring	Link every suggested rewrite to an original resume bullet or approved evidence.	Reduces fabricated claims.
Human approval workflow	Accept, reject, or edit proposed changes before output.	Keeps the user in control.
Cross-job recurring gaps	Summarize skills missing across several jobs.	Creates a practical learning priority list.

14.3 MVP, Stretch, and Future Scope Classification

The following classification protects the deployment deadline. MVP items are required for the final demonstration. Stretch items are attempted only after the complete multi-job workflow is stable. Future items are documented as roadmap features and must not delay submission.

Feature	Category	Decision and Rationale
Upload/paste one resume	MVP	Core input and already part of the baseline.
Submit multiple job descriptions	MVP	Limit to three jobs to keep latency and UI complexity manageable.
Rank jobs and explain ranking	MVP	Primary new differentiator and essential pitch feature.
Wording/evidence/real-skill gap classification	MVP	Strong uniqueness with manageable implementation effort.
Skill match, ATS analysis, and safe rewrites	MVP	Existing approved core workflow.
Accept/reject/edit changes	MVP	Required for ethical and user-controlled tailoring.
Repeated skill-gap summary	MVP	Can be derived from existing multi-job results without another complex agent.
Basic analysis history	Stretch	Useful but not required for the live core workflow.
Generate simple new resume PDF from a fixed template	Stretch	Attempt only after approved content export is stable; exact original layout will not be preserved.
Basic company-type inference from the pasted posting	Stretch	May classify startup, product, consultancy, or enterprise without live web research.
GitHub public profile and repository analysis	Future Phase 2	Requires API integration, evidence-confidence rules, rate-limit handling, and privacy design.
Career profile chat for newly	Future Phase 2	Requires persistent profile

learned skills		memory and user-confirmation workflow.
Certificates, portfolio, and document evidence ingestion	Future Phase 2	Needs additional parsers and source-verification logic.
Live company profile research	Future Phase 2	Requires web-search integration, citations, freshness controls, and reliability safeguards.
Company-aware resume tailoring	Future Phase 2	Should follow reliable company research rather than infer unsupported facts.
Career intelligence memory	Future Phase 3	Long-term storage of skills, applications, goals, and accepted changes expands privacy and architecture scope.
Learning roadmap from recurring gaps	Future Phase 3	Valuable after enough jobs and verified profile evidence are stored.
Exact automatic modification of the uploaded PDF	Future / Out of MVP	PDF layout preservation is fragile; generate a new template-based document instead.
LinkedIn integration or private chat ingestion	Future / Conditional	Depends on approved APIs, permissions, and privacy controls.
Automatic job application submission	Out of Scope	Introduces legal, ethical, and platform-policy risks.

14.4 Future Architecture Additions

- Candidate Profile Agent: converts user-approved resume, GitHub, certificate, and career-chat evidence into a structured skill passport.
- Job Ranking Service: compares normalized per-job results and identifies best immediate fit, growth opportunity, and highest-risk application.
- Company Intelligence Agent: gathers cited public company context and recommends relevant emphasis for the selected role.
- Career Memory Service: stores skill evidence, confidence, source, last-used date, and confirmation status.
- Learning Roadmap Agent: converts repeated real skill gaps into prioritized learning and portfolio actions.

15. Milestone Plan

The plan prioritizes an end-to-end vertical slice early, then improves reliability, UX, and presentation readiness. Dates may be adjusted by one day, but the deployment target must remain 9-10 August 2026.

Phase / Dates	Goal	Deliverables	Exit Criteria
Phase 1: 31 Jul	Scope freeze and revised design	Updated SRS; MVP/future split; multi-job schemas; ranking formula; wireframes	No additional MVP features accepted
Phase 2: 1-2	Backend and	Express API; PDF parser; provider	One resume and

Aug	parsing foundation	adapter; normalized resume and multi-job inputs	three jobs reach the backend
Phase 3: 3-5 Aug	Per-job agent analysis	Supervisor; skill, ATS, and rewrite workers; schemas; anti-fabrication validation	One job produces a complete validated report
Phase 4: 6 Aug	Cross-job intelligence	Deterministic ranking; recommendation labels; three-way gap classification; recurring gaps	Three jobs are ranked consistently
Phase 5: 7-8 Aug	Frontend integration	Multi-job input; ranking dashboard; selected-job detail; accept/reject/edit changes	Complete end-to-end journey works
Phase 6: 9-10 Aug	Deployment and stabilization	Production config; free provider/local fallback; error handling; demo dataset	Deployment or documented demo mode is available
Phase 7: 11-12 Aug	Testing and documentation	Unit/API tests; edge cases; README; updated diagrams; privacy notes	Core tests pass and limitations are documented
Phase 8: 13 Aug	Presentation rehearsal	Slides; pitch; screenshots; backup results; final bug fixes	Stable rehearsal completed
14 Aug	Final submission	Source code; updated SRS; presentation; deployed link; demo evidence	Submission package complete

16. Risks and Mitigation Plan

Risk	Impact	Mitigation
Free API quota or policy changes	High	Use provider abstraction; maintain Ollama local fallback; cache demo outputs only for backup, not as live result.
Local model is slow or hardware-limited	High	Use a smaller quantized model; reduce prompt size; run workers sequentially if memory is constrained.
Invalid or inconsistent JSON	High	Use strict schemas, low temperature, parser repair, and one retry.
Fabricated rewrites	High	Require evidence IDs; run deterministic checks; display risk flags; never auto-apply content.
PDF extraction fails	Medium	Support pasted text as guaranteed fallback; limit MVP to text-based PDFs.
Deployment cannot access local Ollama	High	Use an optional free-tier API for hosted mode or present a documented local-backend mode.
Scope becomes too large	High	Freeze the MVP at three jobs, three core workers, deterministic ranking, gap classification, and approval workflow. Move GitHub, company research, career memory, and exact PDF editing to future releases.
Agent latency harms presentation	Medium	Parallelize independent workers; show progress; use a short demo input.
Scoring appears arbitrary	Medium	Publish weights and evidence; compute in code; avoid hiring probability claims.

Resume privacy concerns	Medium	Minimize retention; avoid logging content; provide delete or guest mode.
One worker fails during demo	Medium	Return partial results; show retry button; keep a tested backup dataset.
Time lost on visual polish	Medium	Use reusable UI components and prioritize readability over animation.
Multi-job analysis exceeds free quota or latency	High	Limit MVP to three jobs, reuse parsed resume data, control concurrency, and show progress.
Job scores are not comparable	High	Use the same normalized schema, weights, model settings, and deterministic calculation for every job.
External evidence is mistaken for verified expertise	Medium	Future integrations must label evidence as resume-verified, externally detected, or user-declared and require confirmation.
Company research becomes inaccurate or stale	Medium	Future company intelligence must cite sources, show retrieval date, and separate fact from inference.
PDF regeneration damages formatting	Medium	Keep the original immutable; MVP exports approved content, while future template-based PDF generation uses controlled layouts.

18. Recommended Project Structure

```
resume-job-match-analyzer/
├── client/
│   └── src/
│       ├── components/
│       ├── pages/
│       ├── services/
│       └── hooks/
├── server/
│   └── src/
│       ├── agents/
│       ├── prompts/
│       ├── providers/
│       ├── validators/
│       ├── ranking/
│       ├── services/
│       ├── routes/
│       ├── middleware/
│       └── schemas/
├── future/
│   ├── candidate-profile/
│   ├── github-evidence/
│   ├── company-intelligence/
│   └── career-memory/
├── tests/
├── docs/
├── .env.example
└── README.md
```